Лабораторна робота №3

Мета роботи: Сформувати практичні навички створення графічного інтерфейсу користувача (GUI) у ROS2, навчитися інтегрувати GUI з нодами ROS2, реалізувати передачу команд руху через Topic /cmd_vel, а також організувати відображення статусу системи.

Обладнання та програмне забезпечення

- ПК з ОС Windows, віртуальна машина VirtualBox з Ubuntu 22.04 LTS;
- ROS2 Humble Hawksbill, симулятор turtlesim;
- workspace ~/ros2_ws з Лабораторних робіт №1–2 (пакет motion_control);
- Python 3.10, бібліотеки rclpy і PyQt5;
- система збірки colcon, IDE Visual Studio Code.

Хід роботи

Цю лабораторну роботу виконано в тій самій віртуальній машині та тому самому робочому просторі, що й попередні лабораторні роботи. Роботу команд `ros2 node list` і `ros2 topic list` перевірено.

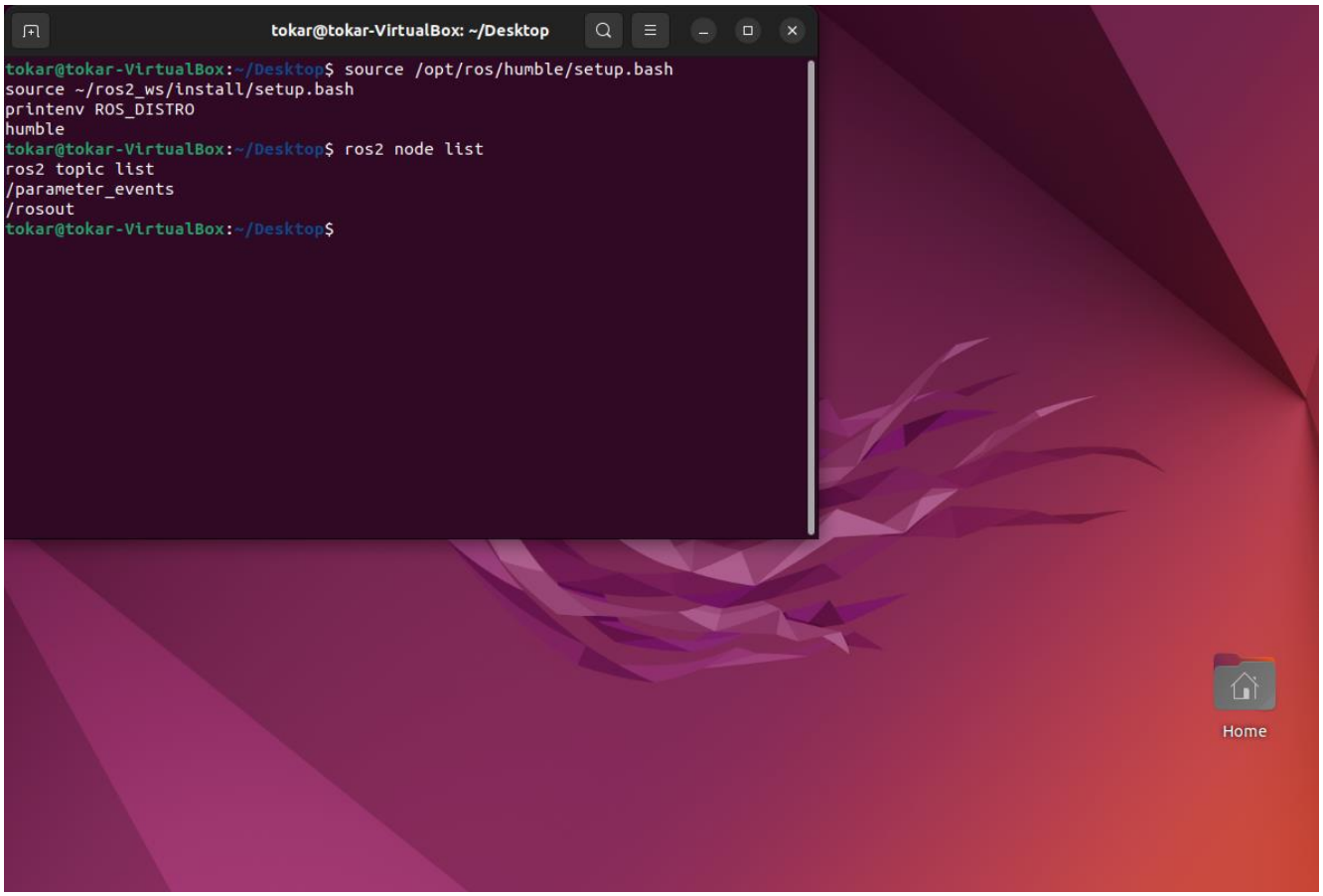


Рисунок 1

Симулятор turtlesim і motion node з Лабораторної роботи №2 було запущено. У виводі присутні обидва вузли (рис. 2).

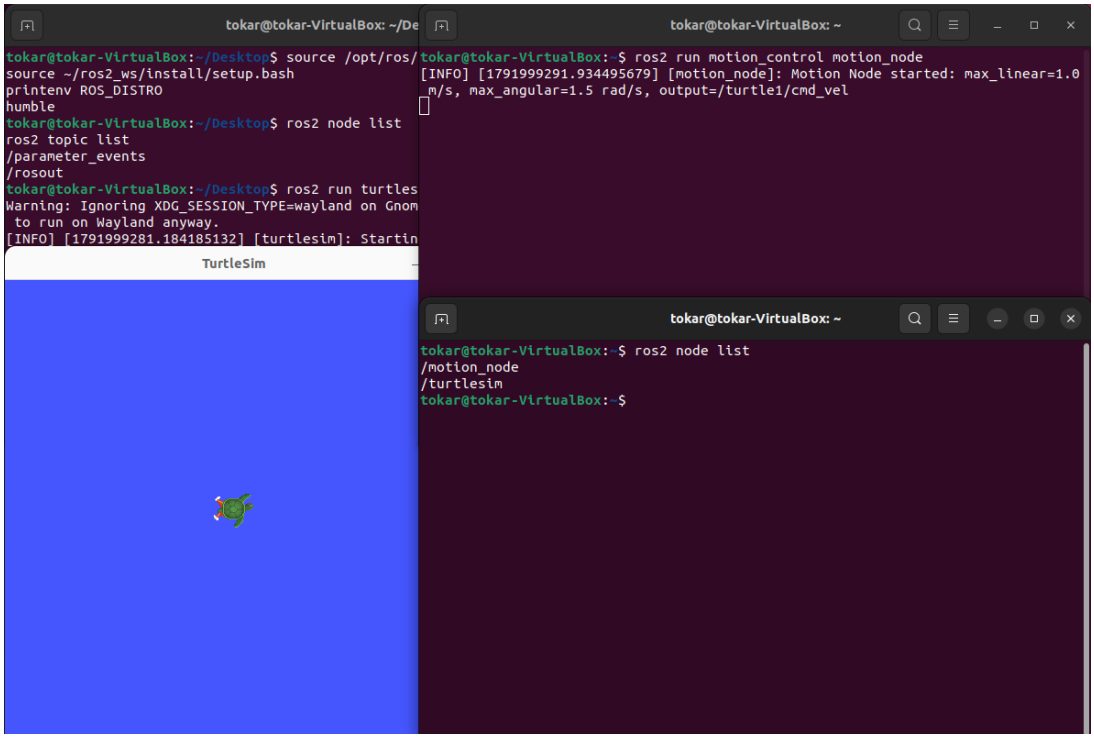


Рисунок 2

У ~/ros2_ws/src створено пакет robot_gui командою.

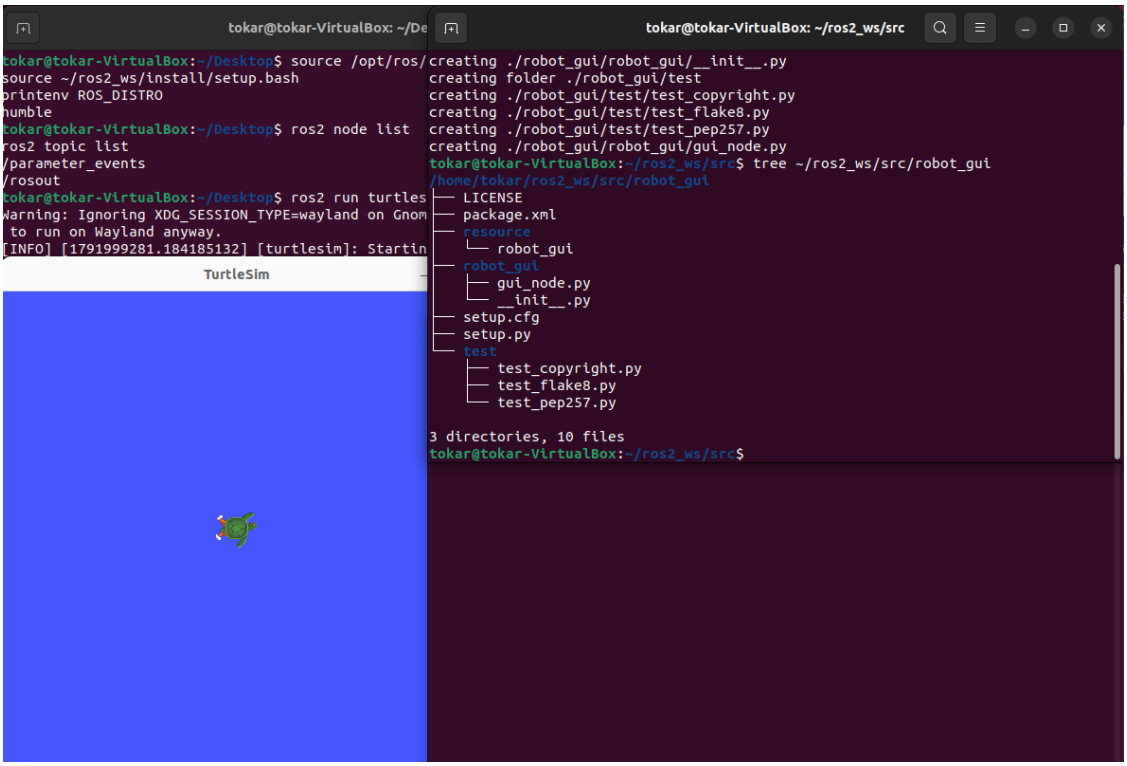


Рисунок 3

Пакет robot_gui зібрано за допомогою команди colcon build --packages-select robot_gui. Правильність встановлення перевірено командами ros2 pkg list і ros2 pkg executables robot_gui (рис. 4).

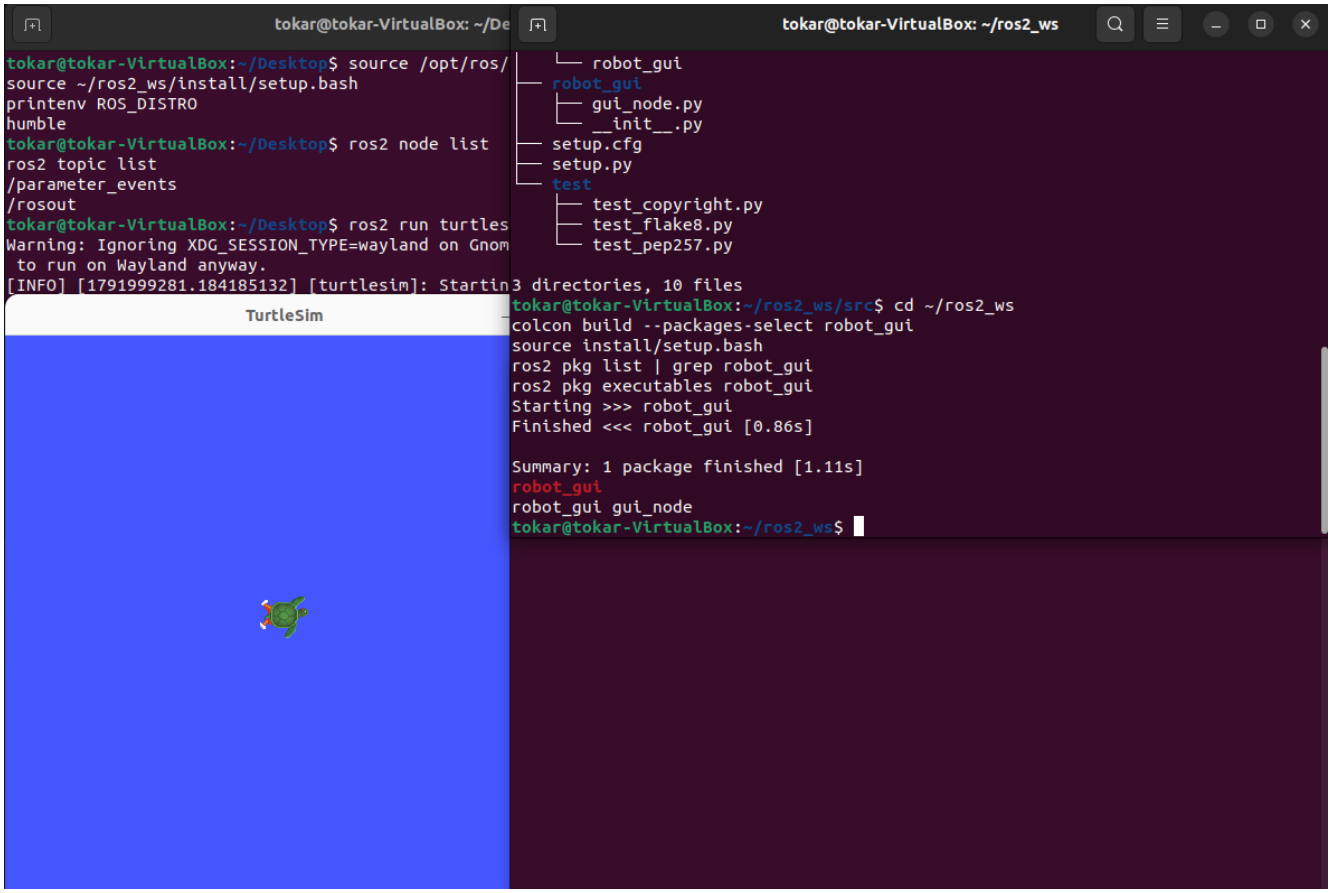


Рисунок 4

Реалізація GUI-вузла

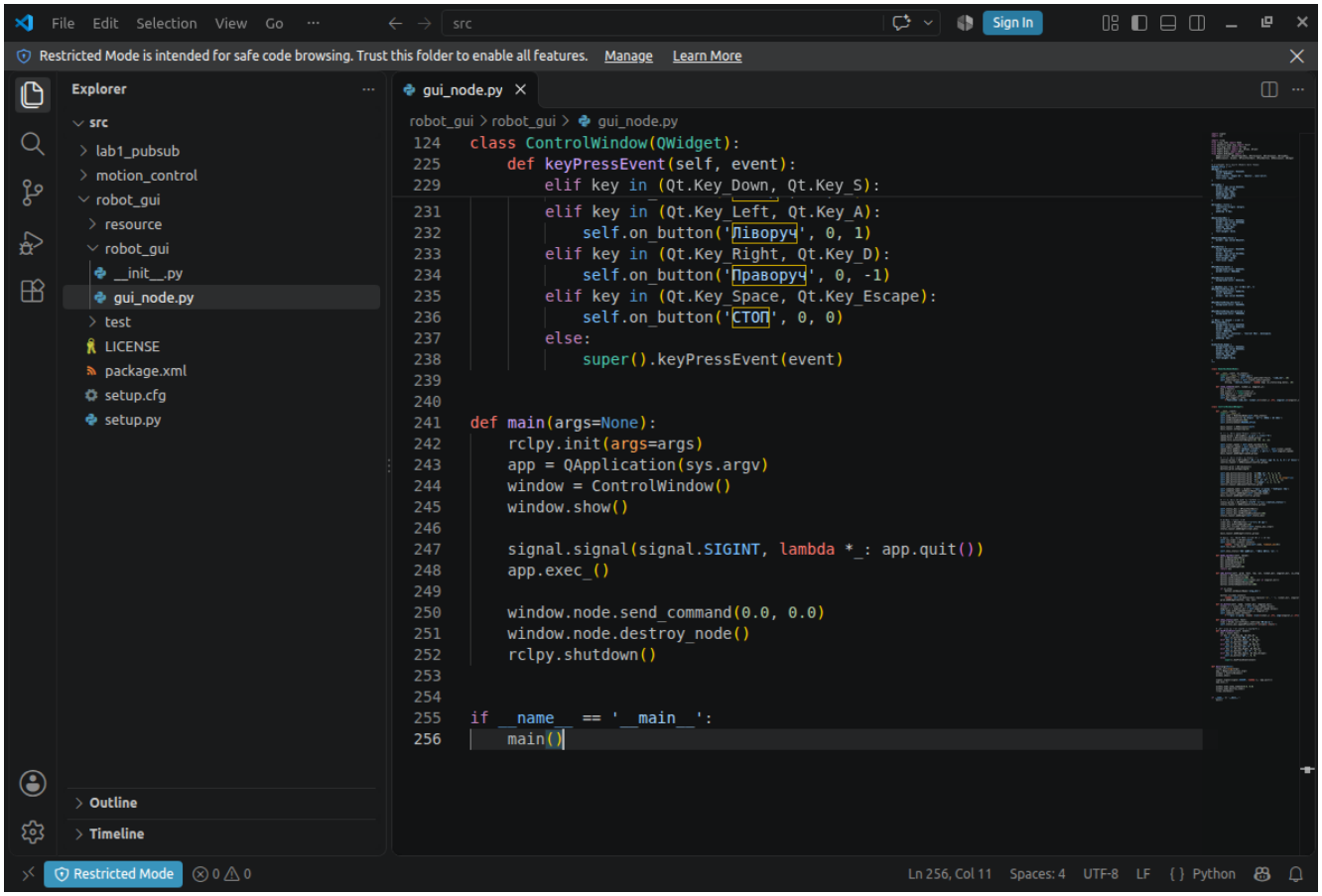
GUI-вузол реалізовано у вигляді двох класів:

1. RobotGuiNode — публікує команди Twist у топік /cmd_vel, підписується на статус системи /motion_status (тип String) і має метод send_command для надсилання значень швидкості.
2. ControlWindow — вікно на основі PyQt5, яке містить:
    - поля для задання лінійної та кутової швидкостей (QDoubleSpinBox);
    - п’ять кнопок руху: «Вперед», «Назад», «Ліворуч», «Праворуч», «Стоп»;
    - текстове поле для виведення повідомлень із /motion_status;
    - мітку з останньою надісланою командою.

Під час натискання кнопки метод on_button формує вектор швидкості (linear.x, angular.z) відповідно до заданих параметрів і передає його вузлу. Завдяки

розділенню коду на два класи логіку зв’язку з ROS2 відокремлено від графічного інтерфейсу.

Код створення GUI:




```
124  class ControlWindow(QWidget):
225      def keyPressEvent(self, event):
229          elif key in (Qt.Key_Down, Qt.Key_S):
231          elif key in (Qt.Key_Left, Qt.Key_A):
232              self.on_button('Ліворуч', 0, 1)
233          elif key in (Qt.Key_Right, Qt.Key_D):
234              self.on_button('Праворуч', 0, -1)
235          elif key in (Qt.Key_Space, Qt.Key_Escape):
236              self.on_button('СТОП', 0, 0)
237          else:
238              super().keyPressEvent(event)
239
240
241  def main(args=None):
242      rclpy.init(args=args)
243      app = QApplication(sys.argv)
244      window = ControlWindow()
245      window.show()
246
247      signal.signal(signal.SIGINT, lambda *_: app.quit())
248      app.exec_()
249
250      window.node.send_command(0.0, 0.0)
251      window.node.destroy_node()
252      rclpy.shutdown()
253
254
255  if __name__ == '__main__':
256      main()
```

Рисунок 5

```
class ControlWindow(QWidget):


    def __init__(self):
        super().__init__()
        self.node = RobotGuiNode(self.show_status)
        self.setWindowTitle('Керування роботом (ROS2 / Qt GUI)')
        self.setMinimumWidth(460)
        self.setStyleSheet(MODERN_STYLE)


        main_layout = QVBoxLayout(self)
        main_layout.setSpacing(12)
```

```
        speed_group = QGroupBox('Параметри швидкості')
        speed_form = QFormLayout(speed_group)
        speed_form.setContentsMargins(15, 15, 15, 15)

        self.linear_speed = self.make_spinbox(0.5)
        self.angular_speed = self.make_spinbox(1.0)
        speed_form.addRow('Лінійна швидкість (м/с):', self.linear_speed)
        speed_form.addRow('Кутова швидкість (рад/с):', self.angular_speed)
        main_layout.addWidget(speed_group)

        control_group = QGroupBox('Пульт керування (або W, A, S, D / Стрілки)')
        control_layout = QVBoxLayout(control_group)

        buttons_grid = QGridLayout()
        buttons_grid.setSpacing(8)

        self.add_button(buttons_grid, '▲\nВперед', 0, 1, 1, 0)
        self.add_button(buttons_grid, '□\nЛіворуч', 1, 0, 0, 1)
        self.add_button(buttons_grid, '■\nСТОП', 1, 1, 0, 0, is_stop=True)
        self.add_button(buttons_grid, '□\nПраворуч', 1, 2, 0, -1)
        self.add_button(buttons_grid, '▼\nНазад', 2, 1, -1, 0)
        control_layout.addLayout(buttons_grid)

    self.command_label = QLabel('Остання команда: Очікування дій')
        self.command_label.setObjectName('cmd_badge')
        control_layout.addWidget(self.command_label)
```

```
        main_layout.addWidget(control_group)
status_group = QGroupBox('Статус системи (/motion_status)')
        status_layout = QVBoxLayout(status_group)

        self.status_box = QPlainTextEdit()
        self.status_box.setReadOnly(True)
        self.status_box.setMaximumBlockCount(150)
        status_layout.addWidget(self.status_box)
clear_btn = QPushButton('Очистити журнал')
        clear_btn.setFixedHeight(26)
        clear_btn.clicked.connect(self.status_box.clear)
        status_layout.addWidget(clear_btn)

        main_layout.addWidget(status_group)

        # Таймер для спину ROS2 (кожні 40 мс = 25 Гц)
        self.ros_timer = QTimer(self)
        self.ros_timer.timeout.connect(
            lambda: rclpy.spin_once(self.node, timeout_sec=0))
        self.ros_timer.start(40)

        self.show_status('GUI запущено, топіки підключено.')

    def make_spinbox(self, value):
        box = QDoubleSpinBox()
        box.setRange(0.0, 5.0)
        box.setSingleStep(0.1)
```

```
        box.setValue(value)
        box.setFixedHeight(28)
        return box

    def add_button(self, grid, text, row, col, linear_dir, angular_dir, is_stop=False):
        button = QPushButton(text)
        button.setMinimumSize(100, 55)
        button.setAutoRepeat(bool(linear_dir or angular_dir))
        button.setAutoRepeatDelay(250)
        button.setAutoRepeatInterval(100)
```

Код Publisher:

```
class RobotGuiNode(Node):

    def __init__(self, on_status):
        super().__init__('robot_gui')
        self.publisher_ = self.create_publisher(Twist, '/cmd_vel', 10)
        self.subscription = self.create_subscription(
            String, '/motion_status', lambda msg: on_status(msg.data), 10)

    def send_command(self, linear_x, angular_z):
        msg = Twist()
        msg.linear.x = float(linear_x)
        msg.angular.z = float(angular_z)
        self.publisher_.publish(msg)
        self.get_logger().info(
            f'Published /cmd_vel: linear.x={linear_x:.2f}, angular.z={angular_z:.2f}')
```

Щоб відображати стан motion node з Лабораторної роботи №2, до неї додано publisher для топіка /motion_status. Після опрацювання кожної команди вузол

публікує повідомлення зі статусом — OK (команду передано без змін) або LIMITED (швидкість було обмежено) — разом з фактичними значеннями.

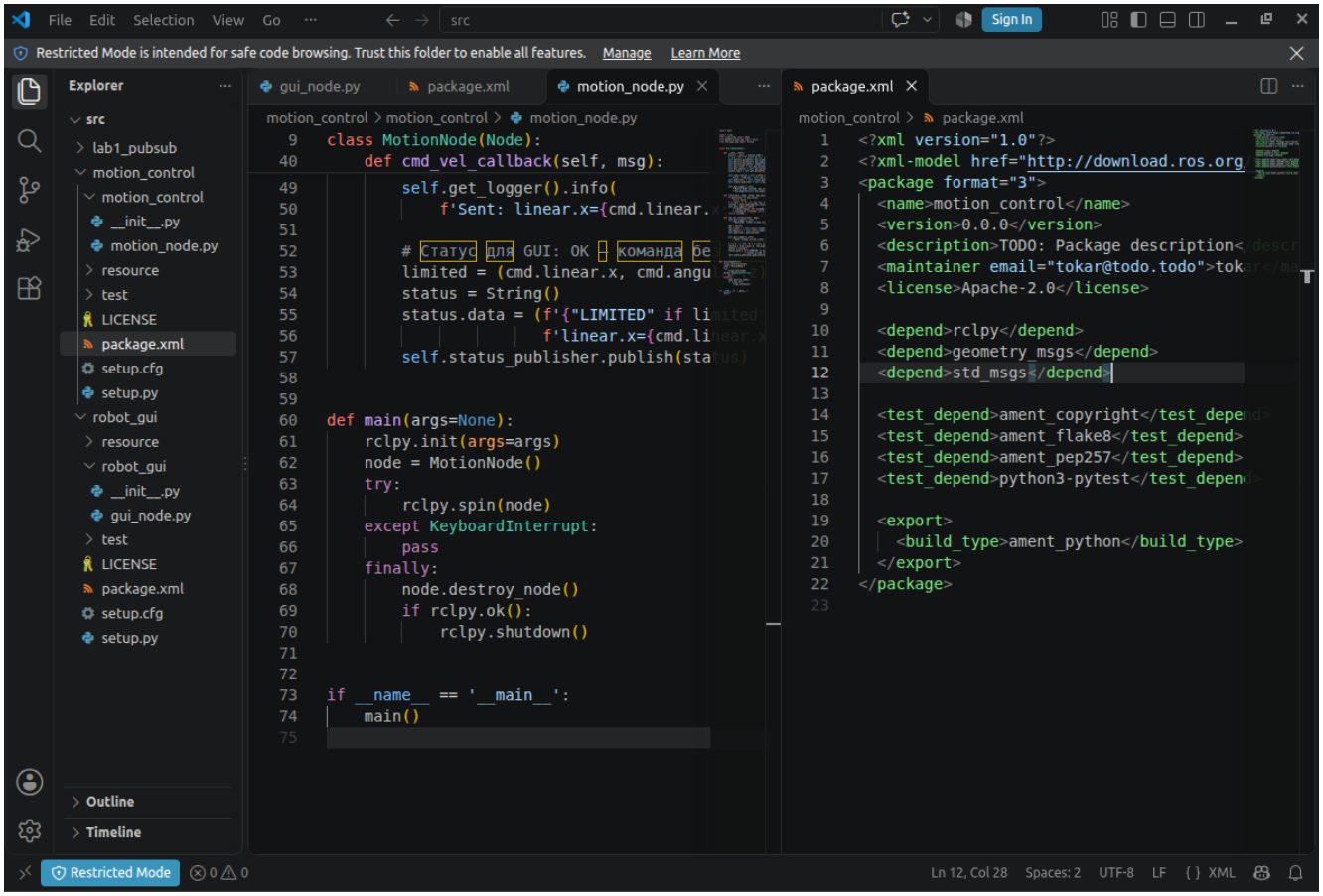


Рисунок 6

Тестування:

GUI-вузол було запущено відповідною командою. Відкрилося вікно керування з полями для задання швидкості, п’ятьма кнопками та полем статусу. До запуску motion node натискання кнопок змінювало рядок «Остання команда», однак черепаха не рухалася, а повідомлення про статус не надходили. Це підтверджує, що GUI не керує роботом безпосередньо, а лише публікує команди в топік /cmd_vel.

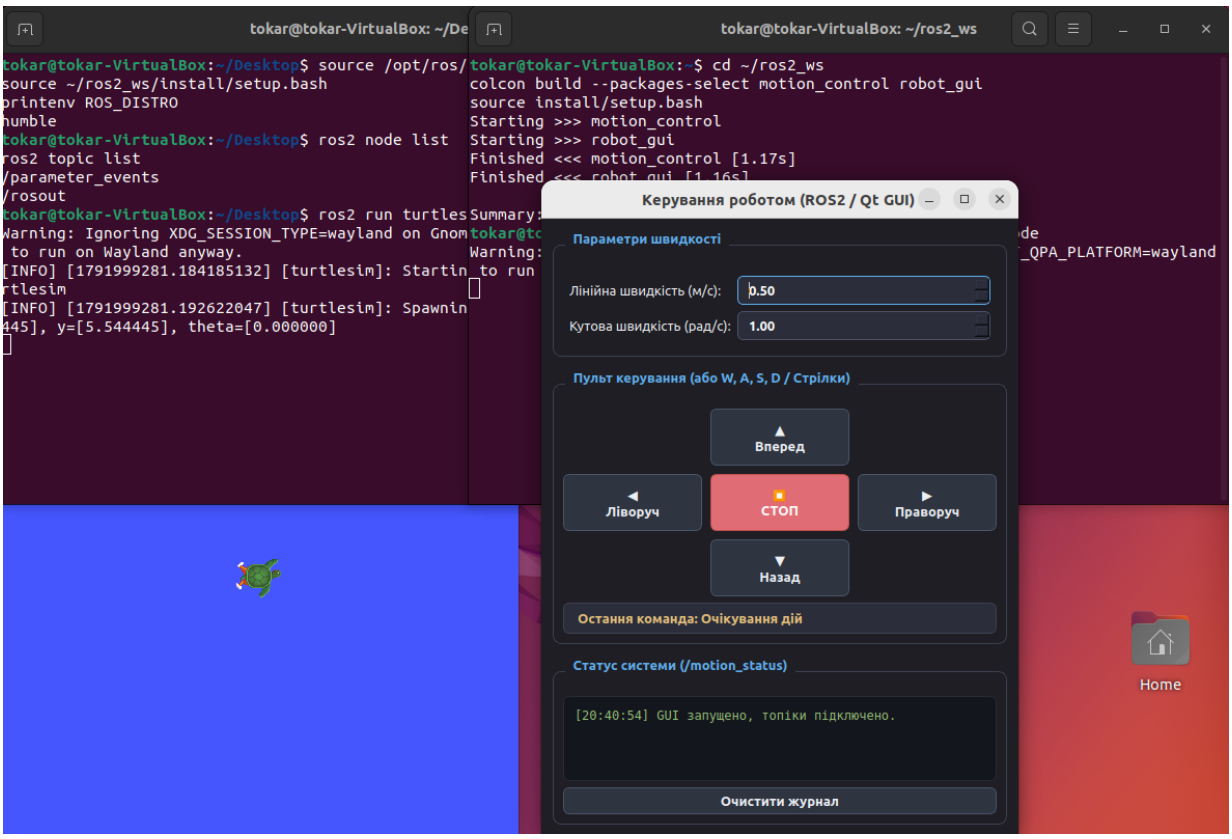


Рисунок 7

Після виконання команди ros2 run motion_control motion_node робот у симуляторі почав реагувати на натискання кнопок. У полі статусу GUI з’явилися повідомлення від motion node із фактичними значеннями швидкості

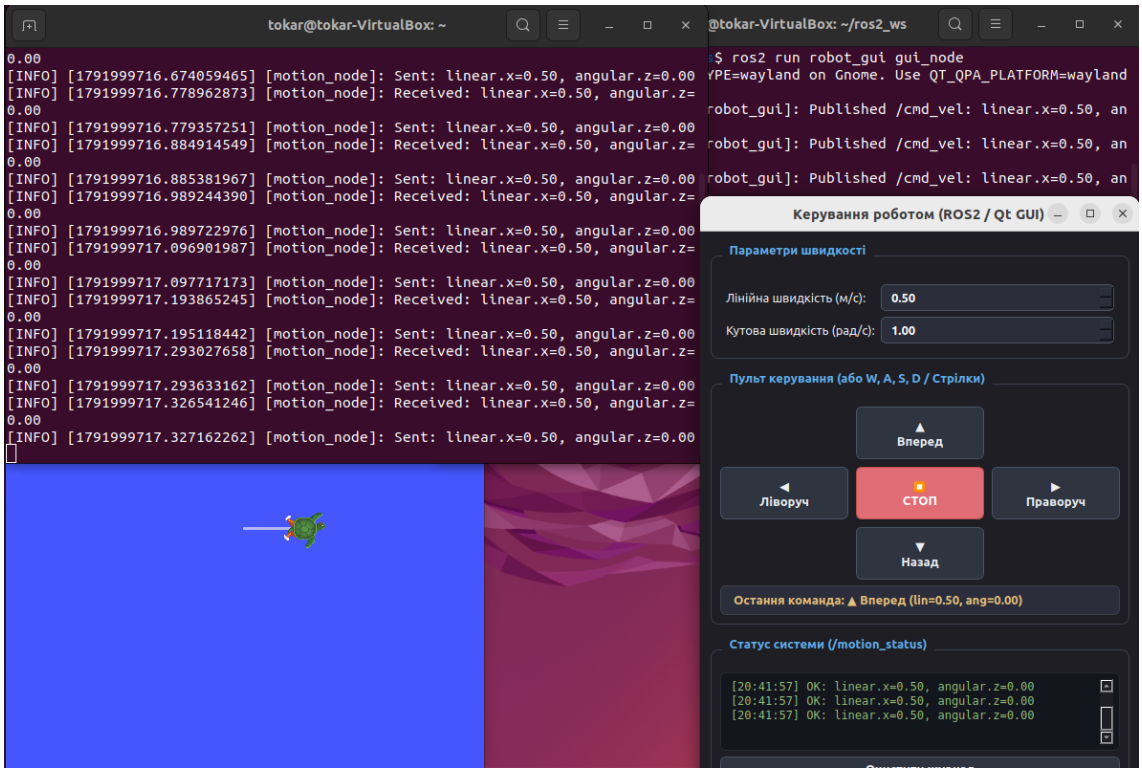


Рисунок 8

Правильність передавання команд перевірено за допомогою команд ros2 topic echo /cmd_vel і ros2 topic echo /motion_status.

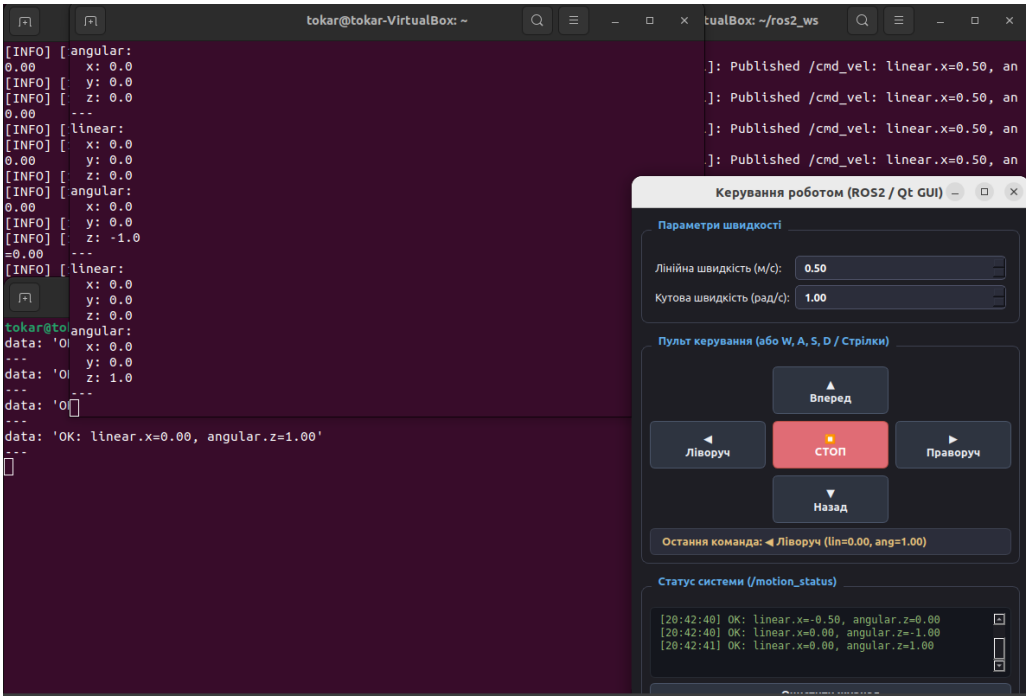
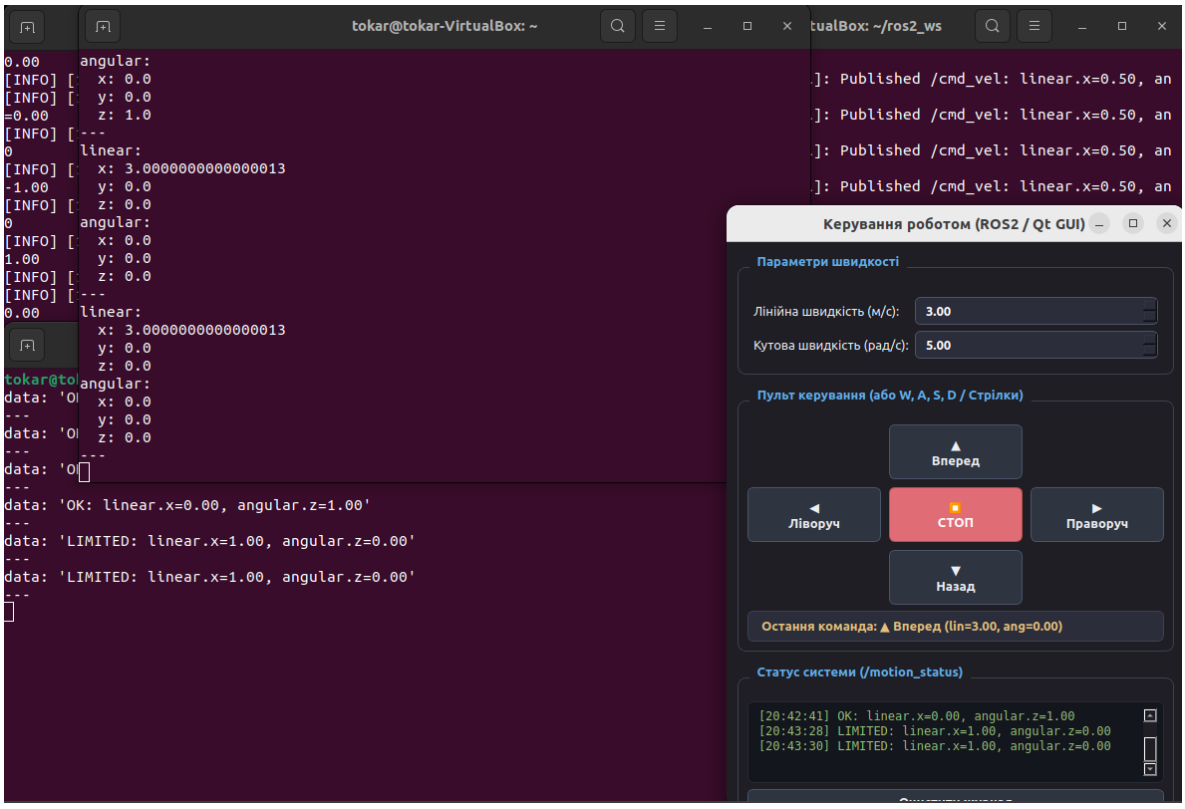


Рисунок 9

Коли в GUI було задано швидкості, що перевищують допустимі пороги motion node (3.0 м/с і 5.0 рад/с), motion node обмежив їх до 1.0 м/с і 0 рад/с. У полі статусу GUI з’явилися повідомлення LIMITED, а в журналі Motion Node — попередження WARN



Висновок:

У результаті виконання лабораторної роботи розроблено Qt-застосунок robot_gui, що є GUI-вузлом і керує роботом через стандартний топік /cmd_vel. За допомогою натискання кнопок генерує повідомлення, які приймає та обробляє motion node, створена в Лабораторній роботі №2. Цю motion node було доопрацьовано: додано публікацію статусу в топік /motion_status, який GUI відображає користувачеві. Для сумісної роботи циклів подій Qt і ROS2 застосовано таймер QTimer, що викликає rclpy.spin_once. Передавання команд перевірено командами ros2 topic echo.